University of Nottingham
UK | CHINA | MALAYSIA

COMP2001/2011: Artificial Intelligence Methods

Lecture 3 – Metaheuristics

Dr Warren G Jackson

The landscape so far…

- Previously we have looked at:
  - Components of heuristic search
  - Simple “low-level” heuristics
  - Hill-climbing heuristics

- Today’s Topics:
  - Metaheuristics
  - Performance comparison of algorithms

- Asynchronous online study:
  - Scheduling (see Moodle)

Metaheuristics

Introduction and Main Components

Definition – Metaheuristics

- A **metaheuristic** is a high-level problem-independent framework providing guidelines/strategies for how to develop heuristic optimisation algorithms.

K. Sörensen and F. Glover. Metaheuristics. In S.I. Gass and M. Fu, editors, Encyclopedia of Operations Research and Management Science, pp 960–970. Springer, New York, 2013. [PDF]

Some Metaheuristics

Classification	Name	Founder(s)
Local Search	Simulated Annealing (SA)	Kirkpatrick (1983)
	Tabu Search (TS)	Glover (1986)
	Guided Local Search (GLS)	Voudouris (1997)
	Iterated Local Search (ILS)	Stutzle (1999)
	Variable Neighbourhood Search (VNS)	Mladenovic (1999)
Population-based	Genetic Algorithm (GA)	Holland (1975)
	Genetic Programming (GP)	Smith (1980)
	Genetic and Evolutionary Computation (EC)	Goldberg (1989)
	Memetic Algorithm (MA)	Moscato (1989)
	Particle Swarm Optimisation (PSO)	Kennedy and Eberhart (1995)
Constructive	Ant Colony Optimisation (ACO)	Dorigo (1992)
	Greedy Randomised Adaptive Search Procedure (GRASP)	Resende (1995)

Types of Metaheuristics

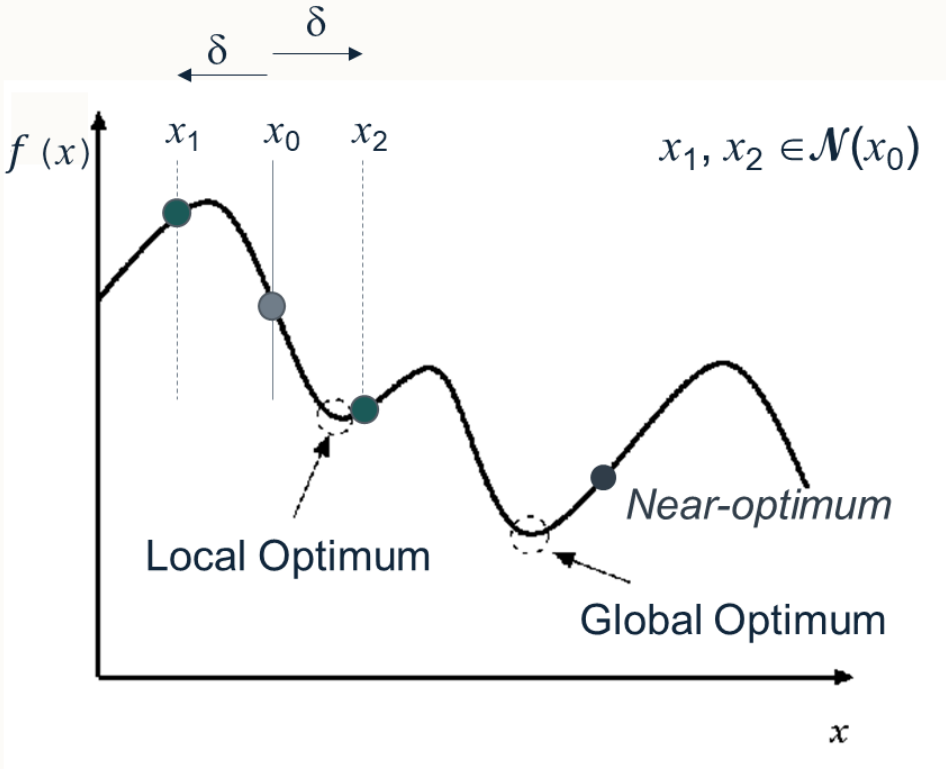
- **Local search** metaheuristics are single-point based trajectory methods that improve a single solution over time.
- **Population-based** metaheuristics evolve a set of solutions over time
- **Constructive** metaheuristics construct new solutions.

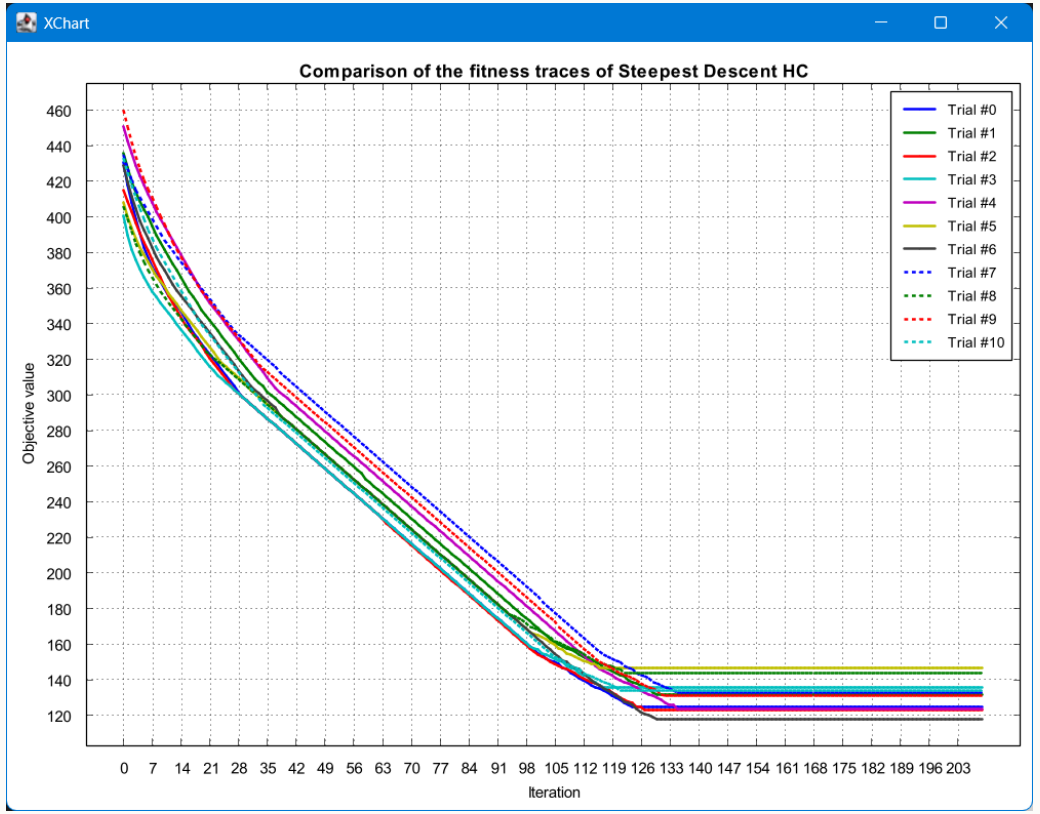
Main Components of Metaheuristics

- Includes the basic components of heuristic search methods seen previously:
  - Solution representation (encoding)
  - Initialisation method
  - Neighbourhood relations (move operators/low-level heuristics)
  - Evaluation function (objective function)
- But now we also need:
  - Mechanism for escaping local optima.
  - Termination condition(s).
  - Multiple move operators. (see e.g. Iterated Local Search).

Mechanisms for Escaping Local Optima 1

- A key issue in heuristic optimisation is how to escape from local optima.





Mechanisms for Escaping Local Optima 2

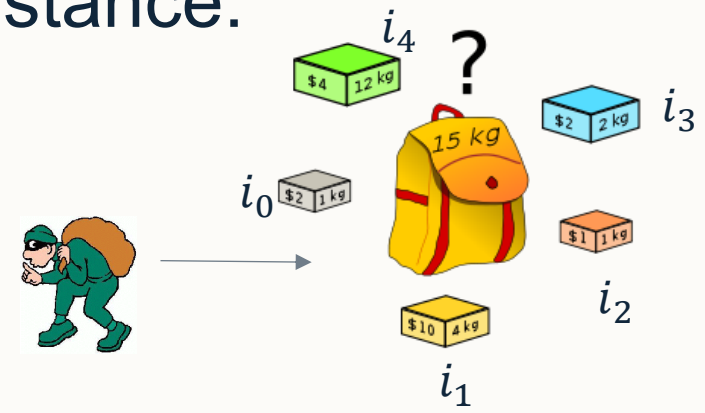
- Change the current solution or restart the search procedure when a local optima is detected.
  - Move to a random solution in the neighbourhood.
  - Generate a random solution (invoke re-initialisation).
- Change the search landscape.
  - Change the objective function.
  - Use a mix of different neighbourhood operators.
- Maintain a memory to prohibit specific moves (e.g. Tabu Search)
- Allow acceptance of non-worsening moves ($\leq$ / $\geq$).
- **Note** neither mechanism is guaranteed to always escape effectively from all local optima.

Termination Criteria

- Exceed a fixed number of the following is exceeded:
  - Iterations
  - Moves
  - Objective function evaluations (we use this in the labs).
  - CPU time (this can be problematic , e.g. Windows 11…)
- Others look at the search progress itself:
  - Consecutive iterations since best solution improved.
  - Evidence that we are at the optimum (e.g. no constraints violated).
  - No feasible solution obtained within a fixed limit.

Feasibility of Solutions

- Not all solutions to problems are feasible; **infeasible solution**.
- Consider the 0/1 Knapsack Problem instance from lecture 2…
- Given the following 0/1 Knapsack Problem instance:
  - $C = 15$
  - $n = 5$
  - $W = \{1.0, 4.0, 1.0, 2.0, 12.0\}$
  - $V = \{2.00, 10.00, 1.00, 2.00, 4.00\}$



Why is the solution <0,1,0,1,1> considered infeasible?

Given the following 0/1 Knapsack Problem instance:

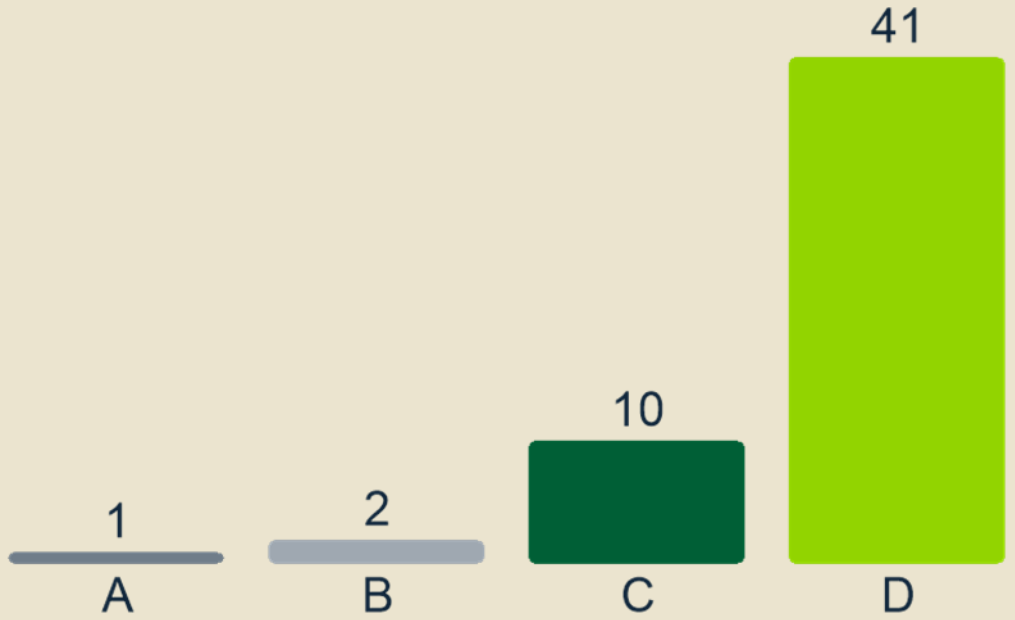
$C = 15$

$n = 5$

$W = \{1.0, 4.0, 1.0, 2.0, 12.0\}$

$V = \{2.00, 10.00, 1.00, 2.00, 4.00\}$

Why is the solution <0,1,0,1,1> considered to be infeasible?



A. Not all items have been packed.

B. The selected items is not the optimal solution.

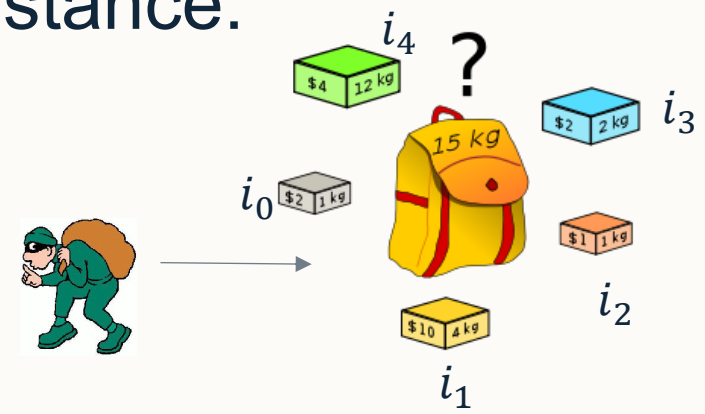
C. The selected items exceeds the volume of the knapsack.

✓D. The selected items exceeds the weight capacity of the knapsack.

The second most common answer was that the selected items exceeds the volume of the knapsack. Remember that 'V' stands for the value of an item. There is no concept of volume in the knapsack problem, and the objective is to maximise the total value of items that are packed.

Feasibility of Solutions

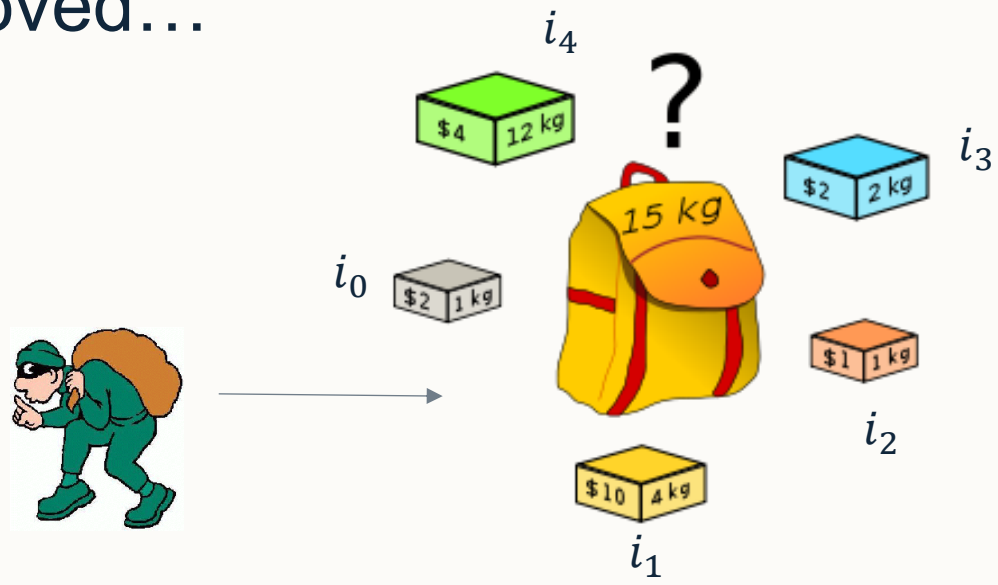
- Not all solutions to problems are feasible; **infeasible solution**.
- Consider the 0/1 Knapsack Problem instance from lecture 2…
- Given the following 0/1 Knapsack Problem instance:
  - $C = 15$
  - $n = 5$
  - $W = \{1.0, 4.0, 1.0, 2.0, 12.0\}$
  - $V = \{2.00, 10.00, 1.00, 2.00, 4.00\}$



- The solution <01011> has a value of 16 but a weight of 18
- The solution <11110> has a value of only 15 but a weight of 8.

Dealing with Infeasibility – Repair

- Problem specific **repair operators**.
  - E.g. remove a random item from the knapsack until the weight does not exceed the limit.
  - The solution <01011> has a value of 16 but a weight of 18 (C=15)
  - Choose an item randomly to be removed…

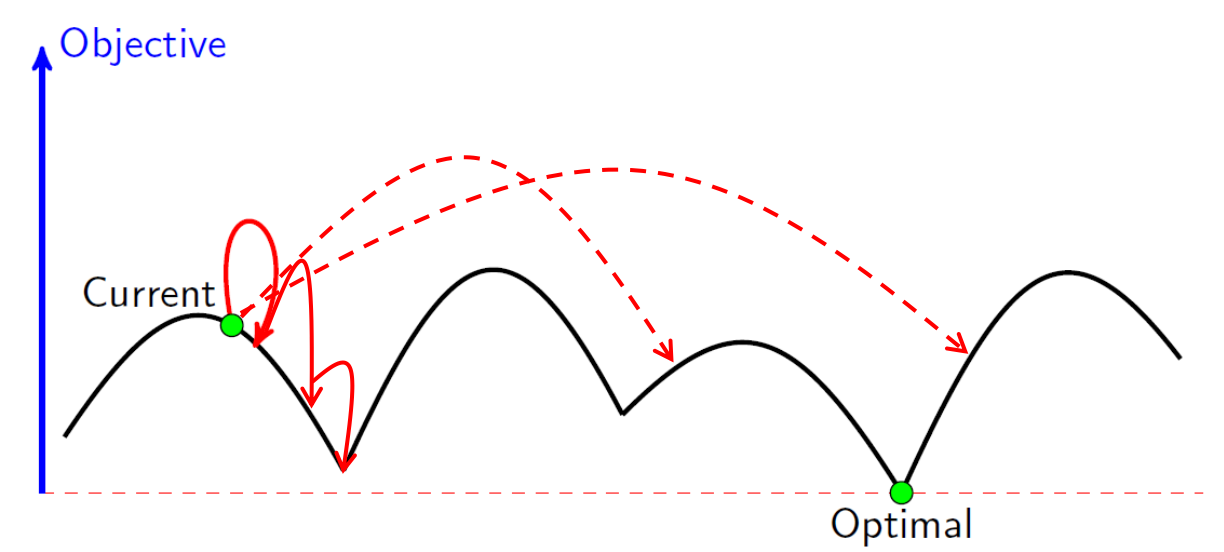


Dealing with Infeasibility – Penalise

- Penalising constraint violations by including them in the objective function.
- Set a fixed penalty value for any infeasible solution.
  - If s is infeasible, then $f'(s) = \min(p_0, \dots, p_4)/2 = \$0.50$
- Set a penalty value based on the level of infeasibility.
  - If s is infeasible then $f'(s) = \min(p_0, \dots, p_4)/\big(2 \cdot (\text{total\_weight} - C)\big)$
- Add a term to the objective function $\gg$ value of any feasible solution. Cost of a solution with any violation **must** be worse than that of the worst feasible solution.
  - $f'(s) = f(s) - 1.01 \cdot sum(p_0, \dots, p_4)$

General Guidelines / The Art of Search

- Effective search techniques provide a balance between **exploration** and **exploitation**, can escape from local optima, and are independent of the initial configuration.



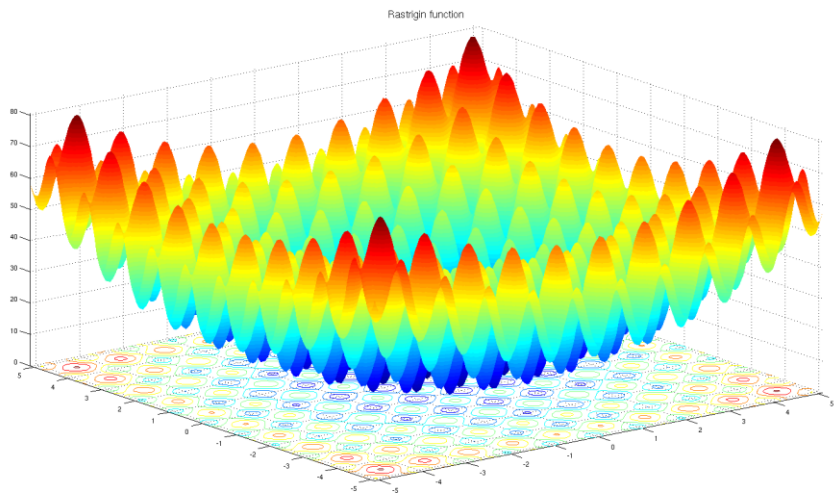
Random Walk ⟷ Hill Climbing

General Template for Local Search Metaheuristics

```
INPUT: s_0 // starting solution

s* ← initialise(s_0) // could improve s_0 or use the original

REPEAT
    s’ ← makeMove(s*, memory) // choose a neighbour s’ of s*
    accept = moveAcceptance(s*, s’, memory) // decide to move to this neighbour or not
    if (accept) s* ← s’ // else reject and “s’ ← s’”

UNTIL (termination conditions are satisfied)
```

A good idea?

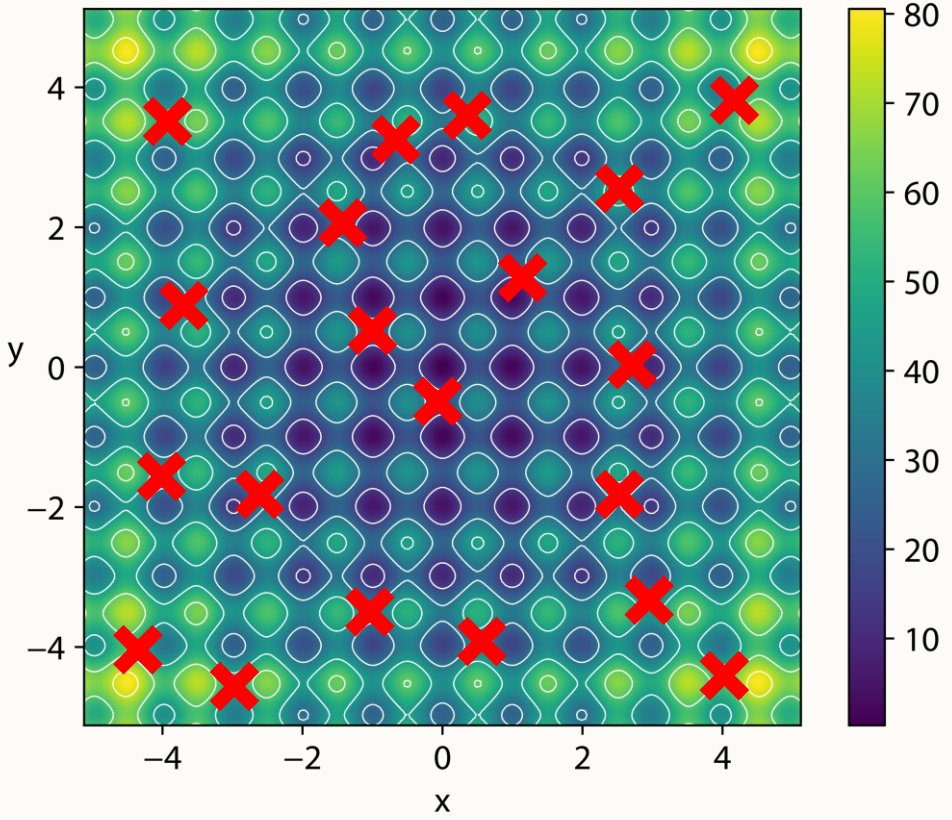
- Use random initialisation in place of the makeMove(…) step?
  - Get a good sample of the search space.
  - Quick if we have an efficient representation.




```
INPUT: s_0 // starting solution

s* ← initialise(s_0) // could improve s_0 or use the original

REPEAT
    s’ ← makeMove(s*, memory) // choose a neighbour s’ of s*
    accept = moveAcceptance(s*, s’, memory) // decide to move to this neighbour or not
    if (accept) s* ← s’ // else reject and “s’ ← s’”

UNTIL (termination conditions are satisfied)
```

A good idea?

- Use random initialisation in place of the makeMove(…) step?
  - Get a good sample of the search space.
  - Quick if we have an efficient representation.



Metaheuristics

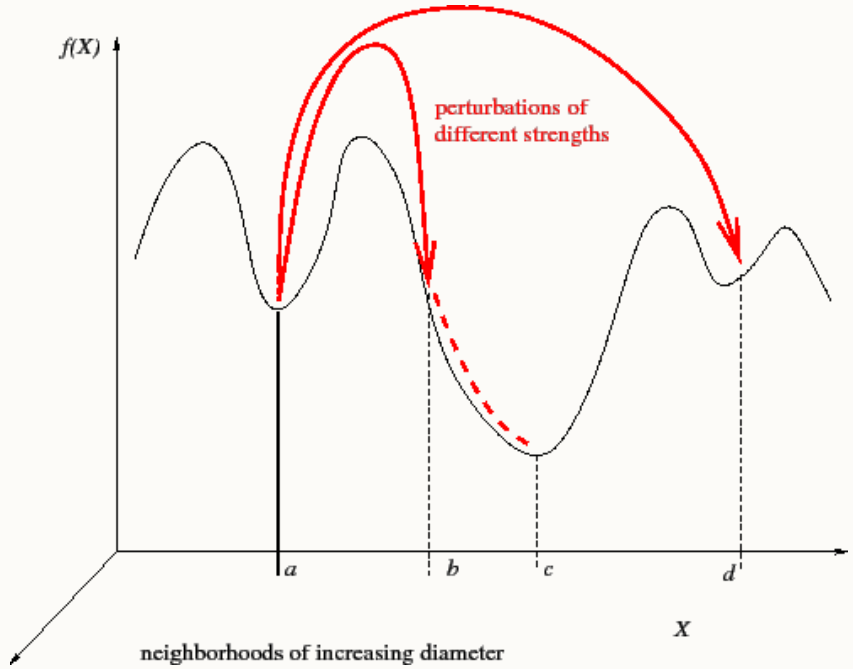
Iterated Local Search

Pseudocode for Iterated Local Search (ILS)

```
s_0 ← generateInitialSolution() // could be random/constructive
s* ← performLocalSearch(s_0) // [OPTIONAL] try to improve the initial solution

REPEAT
    s’ ← performPerturbation(s*, memory) // choose a random neighbour s’ of s*
    s’ ← performHillClimbing(s’, memory) // choose the best neighbour s’ of s’

    accept = moveAcceptance(s*, s’, memory)
    IF (accept) s* ← s’

UNTIL (termination conditions are satisfied)
RETURN s_best
```

Iterated Local Search (ILS)

- Enforces the balance between exploration and exploitation, visiting a sequence of locally optimal solutions.
  - Perturbs the current local optimum (exploration), then
  - Applies hill climbing (exploitation) to get to the new local optimum.
- **Perturbation strength** is crucial:
  - Too weak may lead to cycles.
  - Too strong could lose good properties of the local optima.
- Acceptance criteria.
- Memory (for restarts).



Designing ILS for Solving MAX-SAT

- Representation and objective function follows previous examples.
  - Binary representation
  - Minimisation (total unsatisfied clauses)

- Need to chose each of the following to create an ILS algorithm:
  - GenerateInitialSolution – Random initialisation
  - Perturbation – Random bit flip (random walk)
  - Hill Climbing – Steepest Descent Hill Climbing using a 1-bit-flip neighbourhood relation.
  - Move Acceptance – accepting non-worsening moves $(f(s') \leq f(s^*))$

Applying ILS for Solving MAX-SAT – Initialise Solution

- Given:
  - $\varphi = (x_0 \vee x_1) \wedge (\neg x_3 \vee x_5) \wedge (\neg x_0 \vee x_2) \wedge (x_1 \vee \neg x_5) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_4)$
  - randomSequence $\in [0,5]$ = $[5,0,2,3,2,0,1,1, \dots]$

- Step 1: Generate an initial solution.
  - Total variables is 6 so need a binary string of length 6: < 000000 >
  - Assign each variable to True or False based on equal probabilities.
    - False if random $\in [0,2]$ else True (if random $\in [3,5]$).
  - $s_0 = < 100100 >$ with $f(s_0) = 3$
  - No local search in this example: $s^* \leftarrow s_0$

Applying ILS for Solving MAX-SAT – Loop 1

- Given:
  - $\varphi = (x_0 \vee x_1) \wedge (\neg x_3 \vee x_5) \wedge (\neg x_0 \vee x_2) \wedge (x_1 \vee \neg x_5) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_4)$
  - randomSequence = $[\ldots, 1,1, \ldots]$
  - $s^* = < 100100 >$ with $f(s^*) = 3$

- Step 2 – Repeat loop:
  - $s' \leftarrow < 110100 >$ with $f(s') = 4$ // apply perturbation
  - $s' \leftarrow < 111100 >$ with $f(s') = 1$ // result of applying SDHC
  - $f(s') \leq f(s^*)$, therefore $s^* \leftarrow s'$. // move acceptance

```
REPEAT
    s' ← performPerturbation(s*, memory) //
    s' ← performHillClimbing(s', memory) //

    accept = moveAcceptance(s*, s', memory)
    IF (accept) s* ← s'

UNTIL (termination conditions are satisfied)
RETURN s_best
```

What would be the solution after applying a second loop of Iterated Local Search?

Given:

$\varphi = (x_0 \vee x_1) \wedge (\neg x_3 \vee x_5) \wedge (\neg x_0 \vee x_2) \wedge (x_1 \vee \neg x_5) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_4)$

randomSequence = $[\ldots, 1, \ldots]$

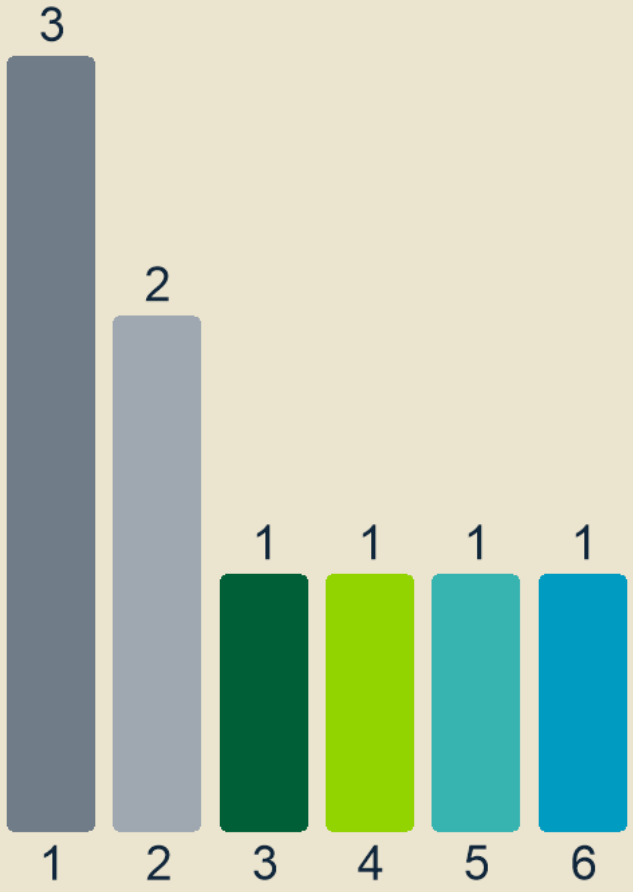
$s^* = < 111100 >$ with $f(s^*) = 1$

What would the solution be after applying another iteration of the main loop of the ILS metaheuristic?

Enter your answer in binary form, e.g. 111100.

```
REPEAT
    s' ← performPerturbation(s*, memory) //
    s' ← performHillClimbing(s', memory) //

    accept = moveAcceptance(s*, s', memory)
    IF (accept) s* ← s'

UNTIL (termination conditions are satisfied)
RETURN s_best
```

Rank	Response
1	111100
2	111110
3	111111
4	111101
5	1111
6	1



Applying ILS for Solving MAX-SAT – Loop 2

- Given:
  - $\varphi = (x_0 \vee x_1) \wedge (\neg x_3 \vee x_5) \wedge (\neg x_0 \vee x_2) \wedge (x_1 \vee \neg x_5) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_4)$
  - randomSequence = $[\ldots, 1, \ldots]$
  - $s^* = < 111100 >$ with $f(s^*) = 1$

- Step 3 – Repeat loop:
  - $s' \leftarrow < 101100 >$ with $f(s') = 1$ // apply perturbation
  - $s' \leftarrow < 101000 >$ with $f(s') = 0$ // result of applying SDHC
  - $f(s') \leq f(s^*)$, therefore $s^* \leftarrow s'$. // move acceptance

- Known global optimum found, return s^*.

```
REPEAT
    s' ← performPerturbation(s*, memory) //
    s' ← performHillClimbing(s', memory) //

    accept = moveAcceptance(s*, s', memory)
    IF (accept) s* ← s'

UNTIL (termination conditions are satisfied)
RETURN s_best
```

Designing Another ILS for Solving MAX-SAT

- GenerateInitialSolution – Random initialisation
- **Perturbation** – n-RandomBitFlips '*intensityOfMutation*' times.
- **Hill Climbing** – DBHC 'depthOfSearch' times.
- Move Acceptance – accepting non-worsening moves ($f(s') \leq f(s^*)$)

- Topic of the lab 3 exercises ☺

Designing ILS for Solving TSP

- Solution representation is permutation encoding; need to choose different components for ILS to be applicable to TSP.
- GenerateInitialSolution
  - Nearest Neighbour
- Perturbation
  - Exchange operator – applied n-times.
- Hill Climbing
  - First improvement with an insertion neighbourhood relation.
- Move Acceptance
  - Accepting improving moves.

Designing Another ILS for Solving TSP

- Solution representation is permutation encoding; need to choose different components for ILS to be applicable to TSP.
- GenerateInitialSolution
  - Random permutation
- Perturbation
  - Inversion operator – applied n-times.
- Hill Climbing
  - DBHC with a pairwise interchange neighbourhood relation.
- Move Acceptance
  - Accepting non-worsening moves.

Guidelines for Designing ILS

- Solution initialisation generally irrelevant for longer budgets.
- Perturbation strength ↔ acceptance criterion interactions can be important and determine the relative balance between exploration/exploitation.
  - E.g. large perturbations only useful if they can be accepted.
- Local search should be fast and effective.
- Ideally choice of perturbation operator should be such that the steps are not easily undone by the local search operator.
- Advanced methods may:
  - Adapt the perturbation strength during the search.
  - Make use of memory to replace the current solution.

Metaheuristics

Tabu Search

Tabu Search (TS)

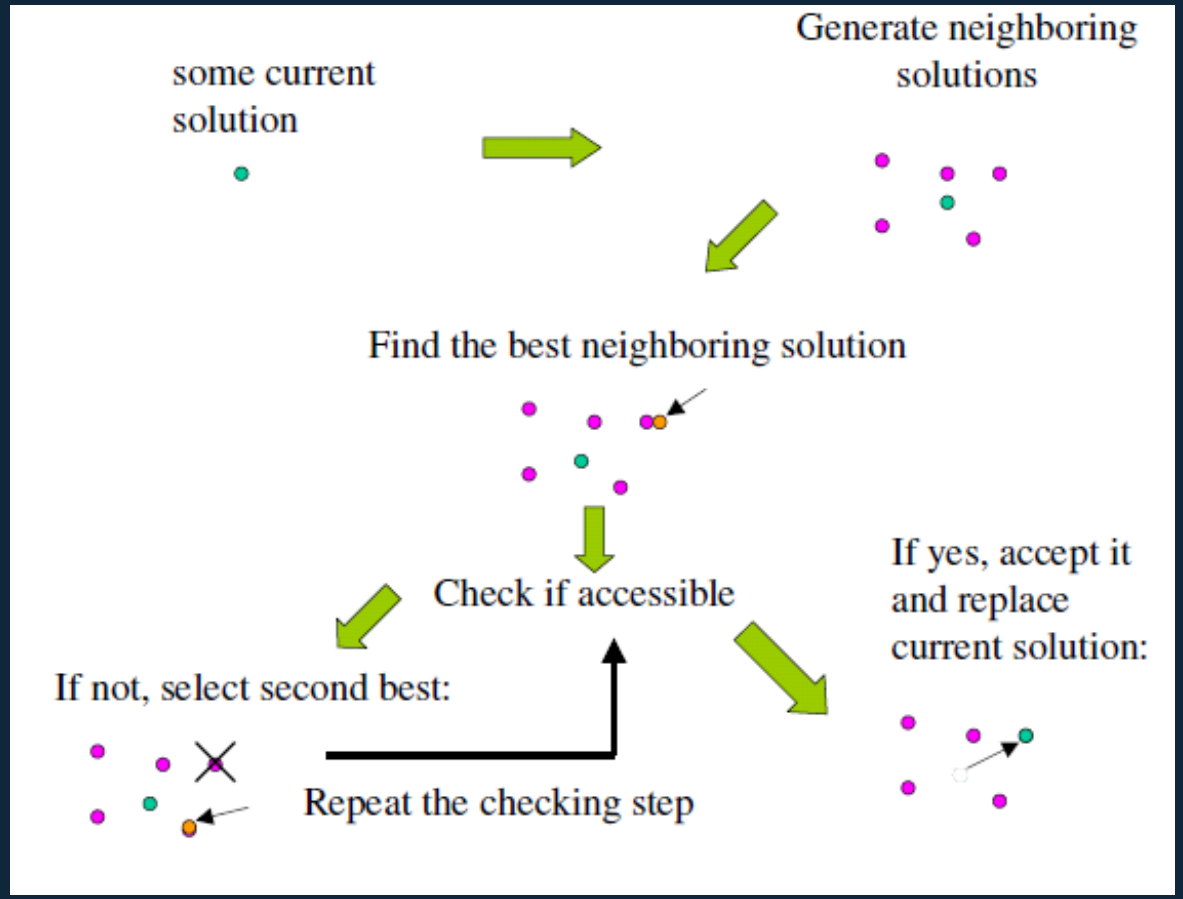
- Proposed by Fred Glover in 1986.
- Novel idea is to maintain a memory/list of tabu/prohibited moves to avoid cycles.

```
s* ← s_0 ← initialiseSolution()

REPEAT
    N’ ← determine non-tabu neighbours of s* // get all neighbours of s* that are not tabu
    s’ ← best(N’) // chooses the best solution from N’

    updateTabuAttributes(s’) // restrict certain neighbourhoods by making them tabu
    s* ← s’ // always accept

UNTIL (termination conditions are satisfied)
RETURN s_best
```

Glover F 1986 Future Paths for Integer Programming and Links to Artificial Intelligence. Computers and Operations Research. Vol. 13, pp. 533-549. [PDF]

Components of Tabu Search

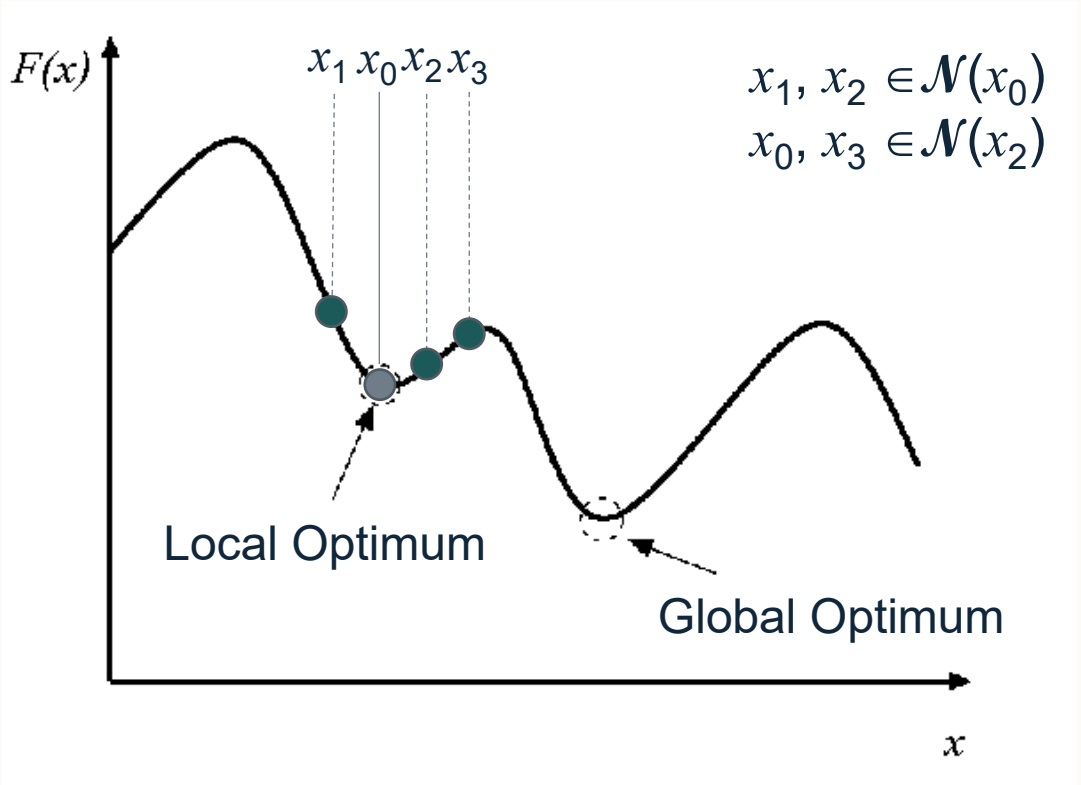
- **Forbidding strategy** controls which moves are in the tabu list.
- **Freeing strategy** controls when moves are removed from the tabu list.
- **Short-term strategy** manages the inter-play between the forbidding and freeing strategies

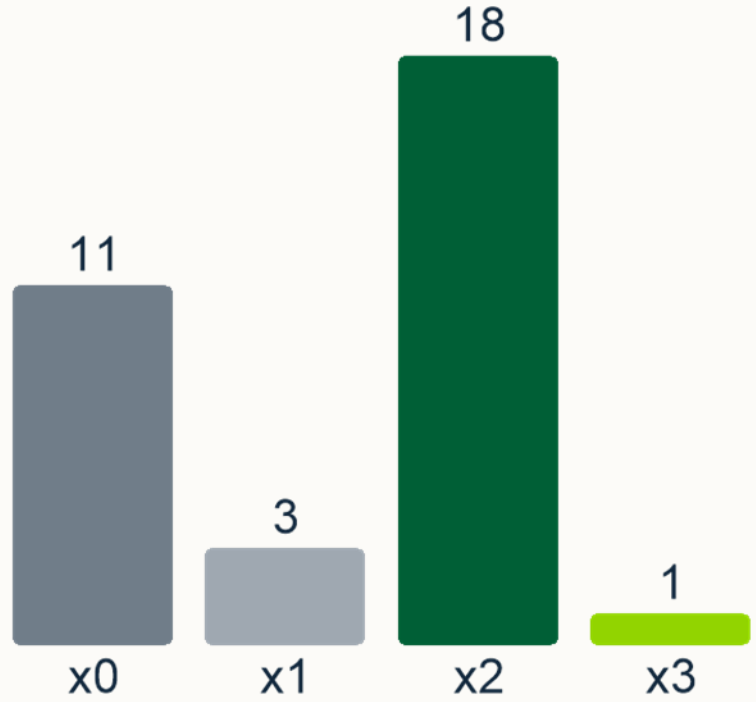


Consider the search landscape on the slide with x_0 being the current solution. Which solution would be *accepted* as the new solution using Tabu Search? You should assume the tabu list is empty.

The correct answer is x_2. The second most common answer was x_0. Remember that in tabu search we must Select a solution from the neighbour of the current solution (so either x_1 or x_2). x_2 is the best neighbouring solution and is not currently prohibited by the tabu list which is currently empty. Tabu search can accept neighbouring solutions even if their cost is worse than that of the current solution.

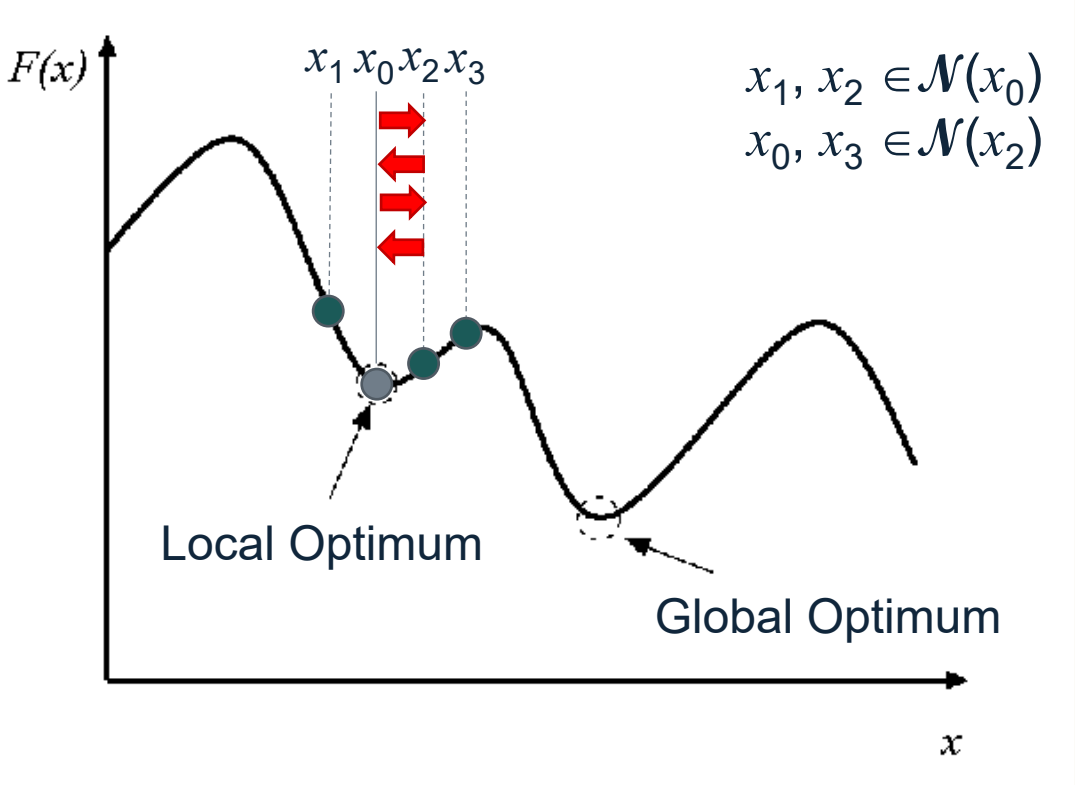
A. x_0
B. x_1
✓C. x_2
D. x_3





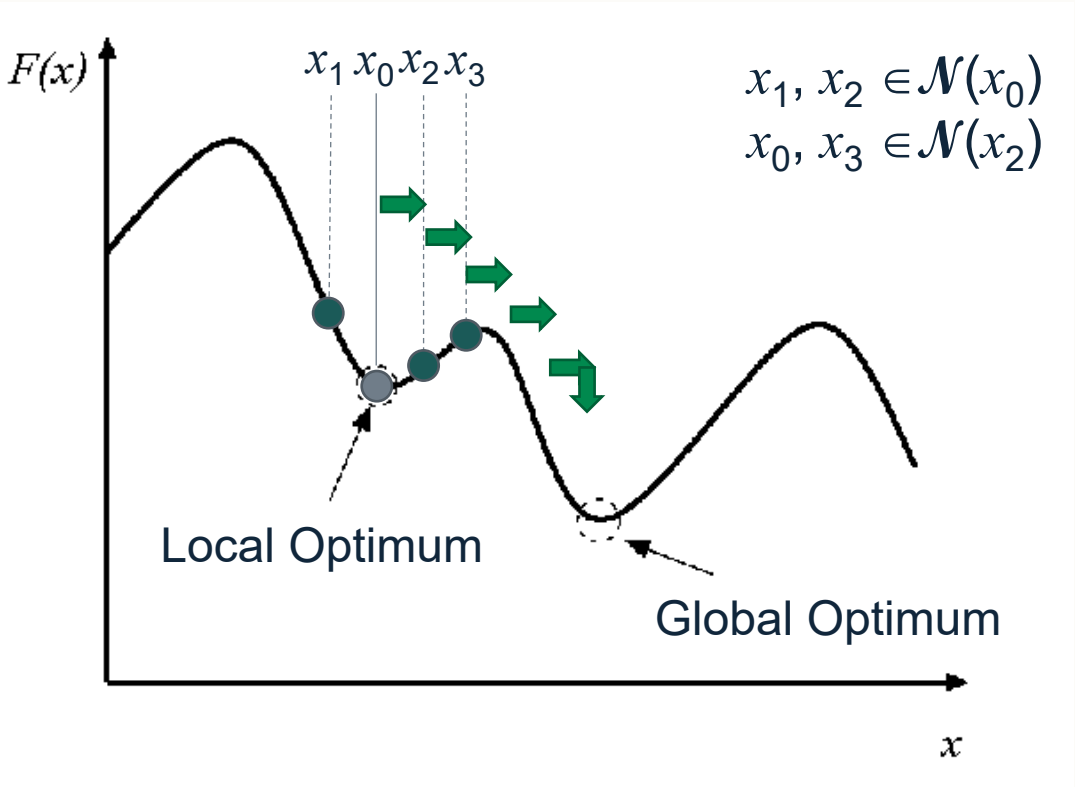
Tabu Search Visually – Part 1

- Without a tabu list, we will become trapped in a local optimum.
- Given an initial solution x_0
- Step 1:
  - Determine all neighbours of x_0
    - $N' = \{x_1, x_2\}$
  - $s^* \leftarrow$ choose the best in N' as x_2
- Step 2:
  - Determine all neighbours of x_2
    - $N' = \{x_0, x_3\}$
  - $s^* \leftarrow$ choose the best in N' as x_0
- The search is stuck moving between x_0 and x_2…



Tabu Search Visually – Part 2

- With a tabu list, we can remember previously visited solutions and consider them as tabu – **note in practice we forbid attributes rather than complete solutions**.
- Given an initial solution x_0 and $T = \emptyset$
- Step 1:
  - Determine admissible neighbours of x_0
    - $N' = \{x_1, x_2\}$
  - $s^* \leftarrow$ choose the best in N' as x_2
  - Add x_0 to the tabu list; $T = \{x_0\}$
- Step 2:
  - Determine admissible neighbours of x_2
    - $N' = \{x_3\}$
  - $s^* \leftarrow$ choose the best in N' as x_3
  - Add x_2 to the tabu list; $T = \{x_0, x_2\}$
- Now we can escape the local optimum! 🎉



Here, T represents the tabu list. We have not introduced the concept of the tabu list length or tabu tenure yet so assume no freeing strategy on this slide.

Fundamentals of Tabu Search

- Exploits a memory to prohibit certain moves to guide the search process, avoiding cycles (local optima/neutral spaces).
- Explores admissible neighbours and selects the best.
- The memory contains solution attributes (as opposed to complete solutions) to stay memory efficient.
- A tabu list stores prohibited moves and is managed by a tabu list length/tabu tenure.
- An **aspiration criteria** is often used to allow the tabu status to be overridden.

Guidelines for Tabu Search

- Tabu tenure (t) needs to be set appropriately.
  - If t is too small we risk cycling.
  - If t is too high, we may overly restrict the search.
- General guidelines for setting t:
  - $t = 7$
  - $t = \sqrt{n}$
- Use of an aspiration criteria that defines an **aspiration level** which accepts tabu moves if the cost of a tabu move is better than this level.

Designing TS for Solving MAX-SAT

- Solution representation is binary encoding; need to choose different components for TS to be applicable to MAX-SAT.
- **Initialisation** – Random initialisation.
- **Neighbourhood** – 1-bit flip operator.
- **Memory** – associate moves as tabu if they have been changed in the previous t steps.

Applying TS for Solving MAX-SAT

- Given:
  - $\varphi = (x_0 \vee x_1) \wedge (\neg x_3 \vee x_5) \wedge (\neg x_0 \vee x_2) \wedge (x_1 \vee \neg x_5) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_4)$
  - $t = 2$
- Assuming we have already ran for several steps, and:
  - Tabu list: $\{x_0, x_2\}$
  - $s^* = < 101100 >$ with $f(s^*) = 1$
  - $N' = \{111100, 101000, 101110, 101101\}$ // 001100 and 100100 are prohibited
    - $f(111100) = 1$
    - $f(101000) = 0$
    - $f(101110) = 1$
    - $f(101101) = 1$
  - $s^* \leftarrow < 101000 >$
  - Update tabu list (dequeue x_0 and enqueue x_3) as $\{x_2, x_3\}$

Note that in practice, depending on the aspiration criterion (if any) we may want to explore the prohibited moves as well in case they are permitted under any overriding criteria. Hence N' would contain all neighbours and we prohibit the selection of the best solution if it is tabu and not overridden by the aspiration criteria.

Performance Comparison of Algorithms

Which algorithm is the best?!

Which algorithm is the best?

- We've seen how we can design various heuristics and metaheuristics, but which is the best?
- We need to be precise:
  - What is meant by the best?
  - Best for what?
- Need to formulate a hypothesis.
  - DBHC finds better solutions than SDHC for solving some specific MAX-SAT problem instance φ_x when given a computational time budget of 1 second.
  - Iterated Local Search with the following configuration […] is better on average than just applying the local search operator when given a computational time budget of 10 minutes.

Important Considerations

- When comparing the performance of algorithms, we must ensure that the comparison is fair (computational budget/nominal time/number of evaluations) must be the same for all algorithms.
- Number of trials is important to derive statistical significance (at a minimum 30)
  - The algorithm **may** perform totally different when containing a stochastic element hence we need to increase our sample size.
- The data used to make the comparisons must be drawn from the same sample.
  - Cannot compare results of DBHC for solving instance φ_x against SDHC solving another instance φ_y.

Which Algorithm, A or B, Performs the Best for Solving Some Problem X – Basic Statistics

- Given the following results obtained after performing 30 trials on problem instance 1 of the problem X, which algorithm performs the best for solving Problem X?

run/trial	A	B
1	1	8
2	1	6
3	0	2
4	1	1
5	0	3
6	2	4
7	0	8
8	1	6
9	1	1
10	0	8
11	1	8
12	2	11
13	1	3
14	2	11
15	1	5

- For each algorithm we can compute some basic stats:
  - Avg. – the mean average objective value over all trials.
  - Std. – the standard deviation associated with the avg.
  - Med. – the median objective value over all trials.
  - Min. – The objective value of the best solution found over all trials.

	Algorithm A				Algorithm B			
Instance	avg.	std.	med.	min.	avg.	std.	med.	min.
Inst1	0.9	0.7	1	0	5.7	3.3	6	1

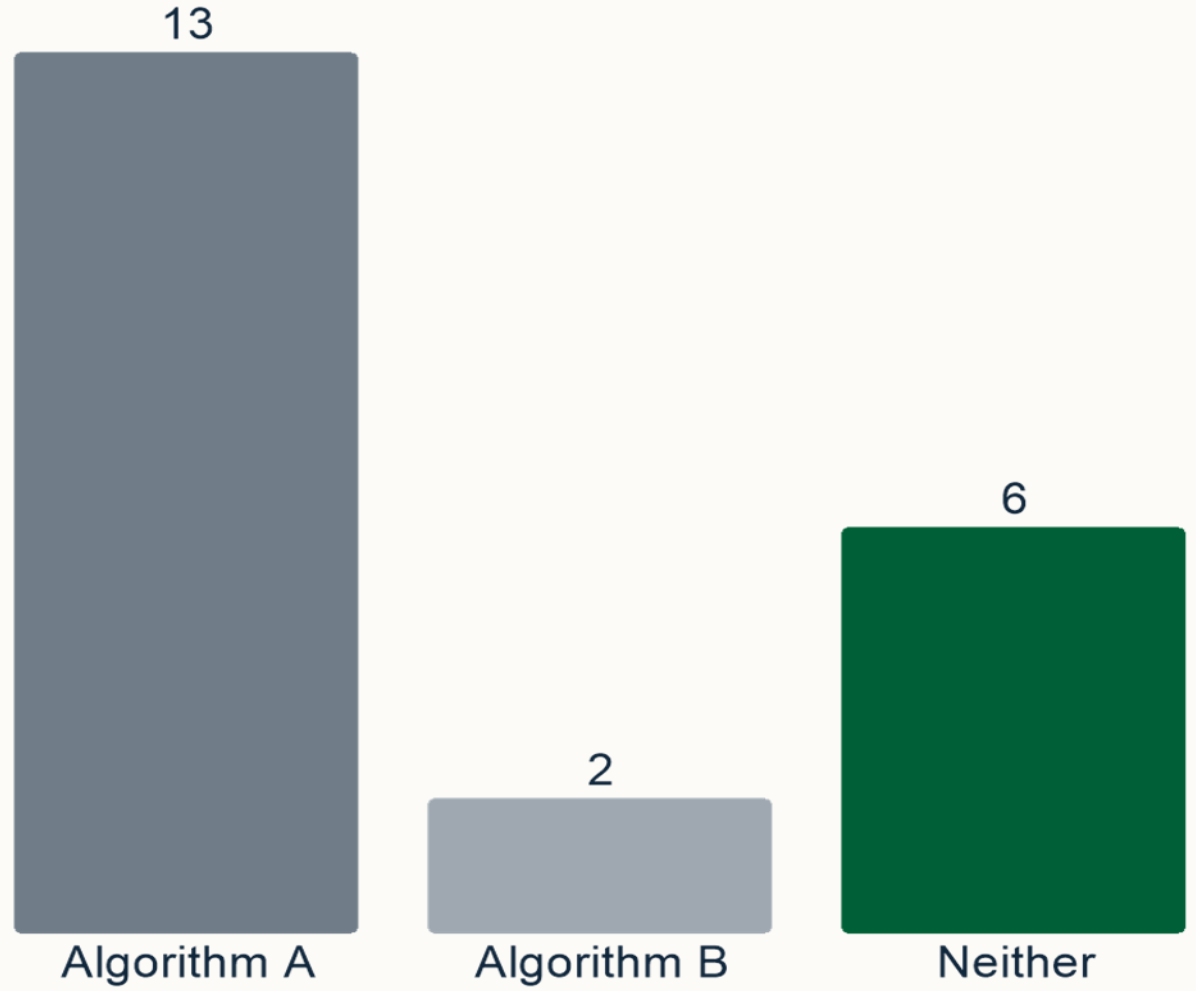
Which algorithm, A or B, performs the best?

A. Algorithm A
B. Algorithm B
C. Neither

	Algorithm A			
Instance	avg.	std.	med.	min.
Inst1	0.9	0.7	1	0

	Algorithm B			
Instance	avg.	std.	med.	min.
Inst1	5.7	3.3	6	1

run/trial	A	B
1	1	8
2	1	6
3	0	2
4	1	1
5	0	3
6	2	4
7	0	8
8	1	6
9	1	1
10	0	8
11	1	8
12	2	11
13	1	3
14	2	11
15	1	5



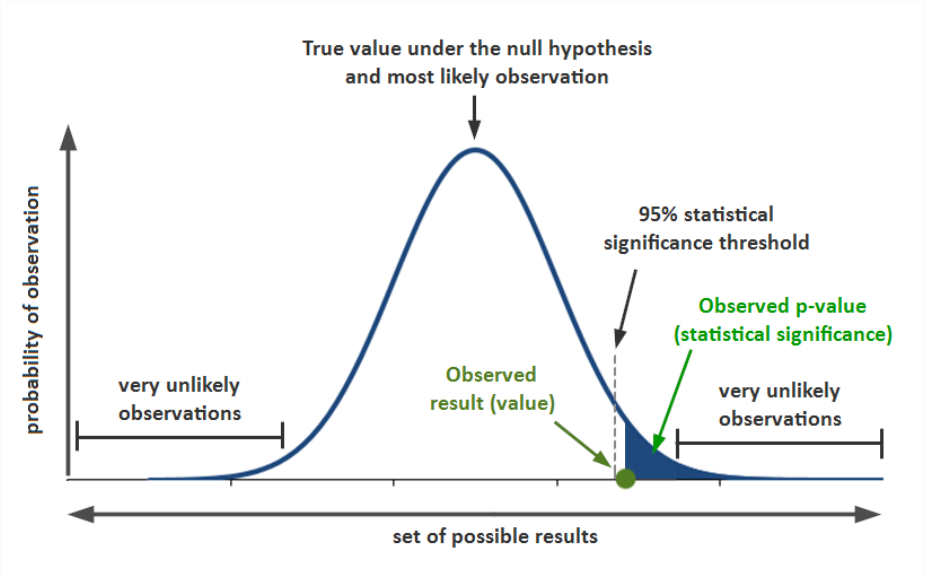
The correct answer is “Neither”. The statement is too vague and assumes algorithm A performs the best in any scenario. The only evidence we have here is for a specific problem instance.

Statistical Testing 1

- Basic stats are ok for an indication but lack significance.
- There are many statistical tests that we can use to conclude stronger statistically significant observations.
- All statistical tests include/compute:
  - A null-hypothesis (H_0) – a statement that we want to test against.
  - A p-value – the probability value which indicates how likely it is that the data would have occurred by random chance, hence, that there is statistical evidence that we can reject the null hypothesis.
  - Rejection of H_0 means that we can accept an alternative hypothesis H_a which is usually the negation of H_0.

Statistical Testing 2

- Typically we use a confidence interval of 95% meaning we are looking for $p \leq 0.05$ to be considered significant.
- There are many statistical procedures and we need to chose the one that is most suitable to our hypothesis/data:
  - Parametric vs non-parametric.
  - Pairwise vs multiple comparisons.
  - One-tailed vs two-tailed testing.
  - Paired (matched) vs unpaired (unmatched) testing.



Which Algorithm, A or B, Performs the Best for Solving Some Problem X – A Statistical Approach

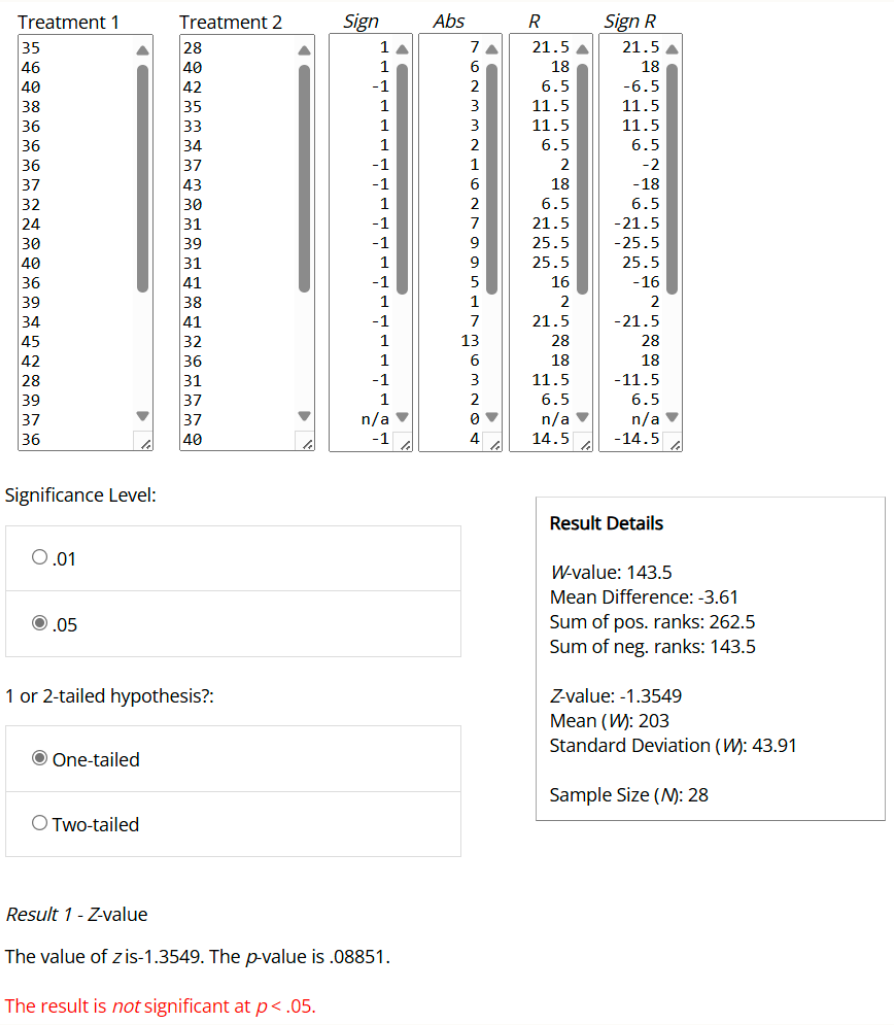
- The (data/results) from heuristic methods (do/does) not come from a normal distribution ⇒ need to use a non-parametric test.
- Our hypothesis: Algorithm A performs better than Algorithm B for solving instance 1 of the problem X.
- A suitable non-parametric test to compare two algorithms with matched data in this case would be a one-tailed **Wilcoxon Signed Rank Test**. Wilcoxon Signed-Rank Test Calculator
- H_0 is that the medians of the two samples are identical.

- Unmatched alternative is the Mann-Whitney U Test Mann-Whitney U Test Calculator

Which Algorithm, A or B, Performs the Best for Solving Some Problem X – A Statistical Approach Example

- We might theorise that SDHC performs better than DBHC based on the better median result, hence perform a one-tailed test with SDHC in treatment 1 and DBHC in treatment 2.
- We find that the p-value is 0.0885 meaning that we fail to reject H_0.
- Hence accept that the medians of the two samples are identical and disprove our hypothesis that SDHC performs better than DBHC for solving a given problem instance.

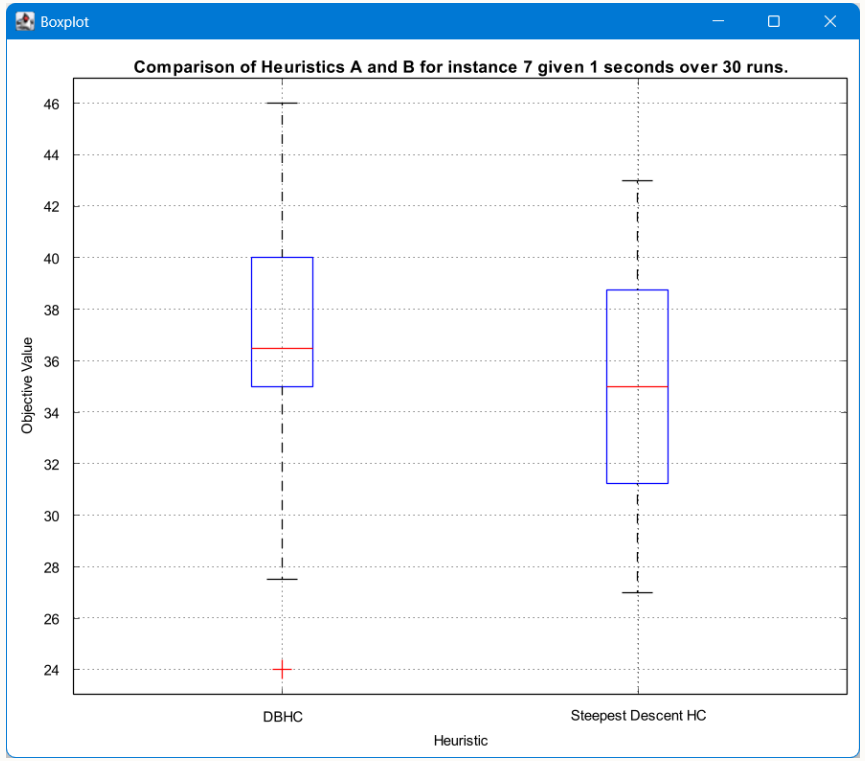
	SDHC					DBHC			
	Avg	Std	Med	Min	vs	Avg	Std	Med	Min
Inst 1	35.2	4.64	35	27	≤	36.6	4.98	36.5	24

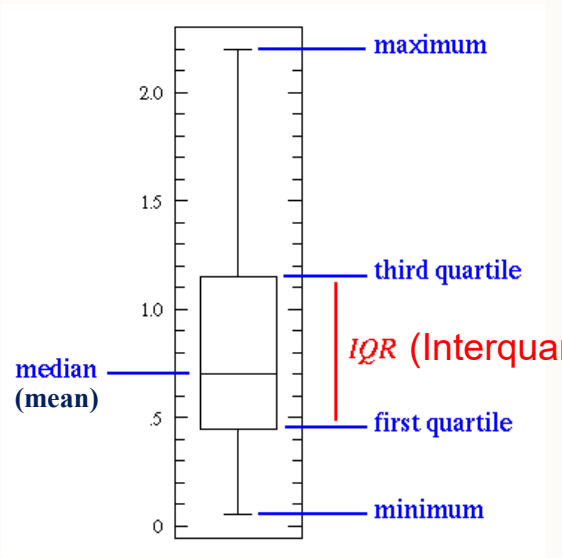


DBHC	SDHC
35	28
46	40
40	42
38	35
36	33
36	34
36	37
37	43
32	30
24	31
30	39
40	31
36	41
39	38
34	41
45	32
42	36
28	31
39	37
37	37
36	40
35	35
43	34
42	35
40	42
29	28
31	27
40	37
37	28
36	34

Comparisons using Boxplots

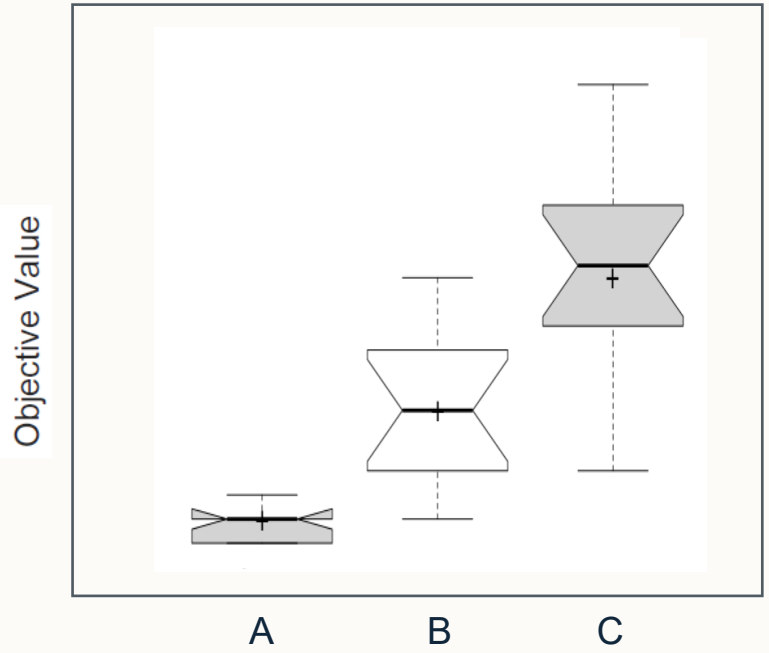
- Boxplots can illustrate relative performances between algorithms.
- Example is that from previous data.





Comparisons using Notched Boxplots

- Notched boxplots allow comparison of confidence intervals for the median of each boxplot.
- If the notches do not overlap, we may conclude the difference is significant.
- We can conclude:
  - $A < B$
  - $B < C$
  - $A < C$



Statistical Tests Continued

- Pairwise testing ok for comparing two algorithms but difficult to generalise any meaning for multiple algorithms.
  - Consider: $A \leq B \leq C \leq D < E$
  - Does A perform significantly better than D? (susceptible to a type 2 Error – false negative)

Nonparametric statistical tests

Type of comparison	Procedures
Pairwise comparisons	Sign test Wilcoxon test
Multiple comparisons ($1 \times N$)	Multiple sign test Friedman test Friedman Aligned ranks Quade test Contrast Estimation
Multiple comparisons ($N \times N$)	Friedman test

Post-hoc procedures

Type of comparison	Procedures
Multiple comparisons ($1 \times N$)	Bonferroni Holm Hochberg Hommel Holland Rom Finner Li
Multiple comparisons ($N \times N$)	Nemenyi Holm Shaffer Bergmann

Comparisons over multiple instances – does some algorithm perform better for solving problem X

- We can perform multiple pairwise tests for each instance, but susceptible to type 1 errors (false positive).
- What if some differences are not significant?
- Assume objective function is maximisation.

	Algorithm A					Algorithm B					Algorithm C			
Instance	avg.	std.	med.	min.	vs.	avg.	std.	med.	min.	vs.	avg.	std.	med.	min.
Inst1	0.9	0.7	1	0	>	5.7	3.3	6	1	>	11.6	4.9	12	3
Inst2	3.1	3.9	2	1	>	21.3	13	12	3	>	44.9	9.8	30	18
Inst3	0.7	0.5	1	0	>	7.1	7.7	3	0	>	26.3	14	13	1
Inst4	1.7	1	1	1	>	5.7	4.3	3	1	>	20	4.6	15	12
Inst5	7.6	0.9	7	7	>	10.4	1.5	8	7	>	15.4	1.7	14	13

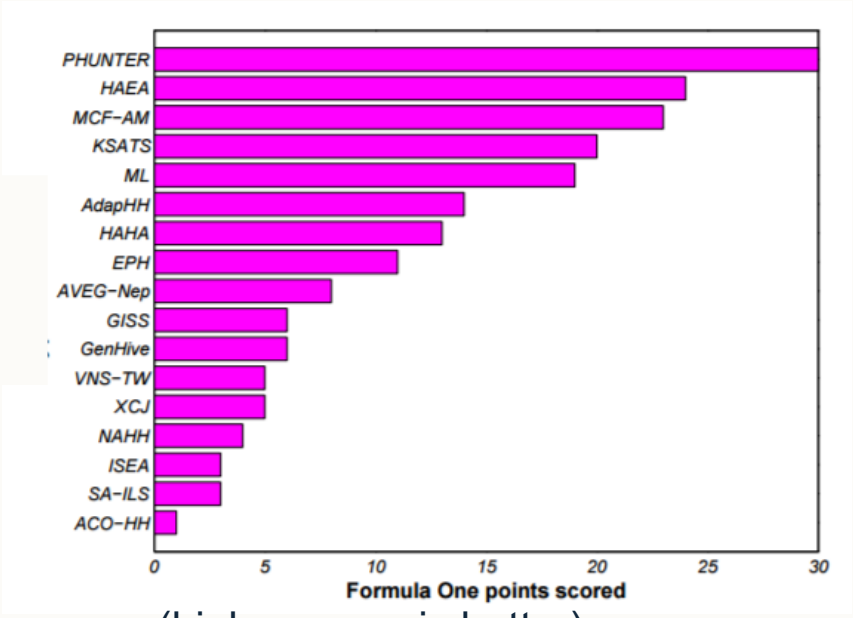
Comparisons over multiple instances – other approaches

- Ranking mechanisms (e.g. F1 score).
- Sum of normalised μ_{norm} scores.
- Multi-way testing/ANOVA with post-hoc analysis

$$f_{norm}(s) = \frac{f(s) - f(s_{best})}{f(s_{worst}) - f(s_{best})}$$

	Cross-domain
IE	17.67
AILLA	15.91
TA	20.04
GD	13.15
AILTA	19.05
NA	29.61
SA	**10.10**
SARH	10.22

(lower score is better)



(higher score is better)

Concluding remarks

- **Office hours** (for me) set as Friday's 3pm-4pm in C87.
- **Lab 1 general feedback** released – review your responses to view.

- Module activities:
  - **Asynchronous online study activity** – Scheduling.
  - Formative self-assessment quiz on Moodle.
  - Computing Session (COMP2001): Friday 13th February CS-A32 4pm – Metaheuristics (Part 1).
  - Next lecture: 17th February 4pm – Move Acceptance.

University of Nottingham
UK | CHINA | MALAYSIA

Thank you

Any questions?